

Enterprise Knowledge-Based Automation Code-Based Architecture & Interview Pack

Read-only analysis

Source: repository code / Terraform / scripts / workflows

No secrets included

Important: `PROJECT.md` still describes "Bedrock only" and "Qdrant as container only." **Live AWS config and application code** use **Cohere embeddings + Groq chat + Qdrant Cloud**, with Bedrock remaining selectable via configuration. This document follows **code + Terraform**.

Table of Contents

1. Executive Summary
2. Technology Stack
3. Complete Architecture
4. Request Flows
5. Master Realistic Example
6. Embedding Model
7. LLM
8. RAG Pipeline
9. Qdrant
10. PostgreSQL / RDS
11. S3
12. Cognito
13. Security
14. ALB
15. Blue-Green Deployment
16. GitHub Actions
17. Terraform & Lifecycle Scripts
18. Redis
19. CloudWatch
20. Secrets Manager (names only)
21. API Endpoint Map
22. Data Lifecycle
23. Destroy → Deploy Lifecycle
24. Interview Speeches

- 25. Interview Q&A
- 26. Final Master Flow
- 27. File / Code References
- 28. NOT IMPLEMENTED

1. Executive Summary

This project is a **multi-tenant enterprise RAG + agentic automation** platform. Companies upload policies/SOPs; authenticated users ask questions; answers are produced by **tenant-filtered vector retrieval + an LLM**, with **citation validation and guardrails**.

Who uses it

- `platform_admin` (tenant `platform`) — creates companies; invites each company's first admin
- company `admin` — manages users **inside its own company only**
- `user` — upload / chat within own tenant

Live AWS AI path (`infra/terraform/envs/dev/main.tf`):

- `AI_PROVIDER=hybrid`
- `EMBED_PROVIDER=cohere` → `embed-multilingual-v3.0` (1024-d)
- `LLM_PROVIDER=groq` → `openai/gpt-oss-20b`
- `QDRANT_COLLECTION=ekba_chunks` , distance **COSINE**, Qdrant Cloud URL

2. Technology Stack

Layer	Actual technology in this repo
Frontend	Next.js 15.1.3, React 19, TypeScript, Tailwind — hosted on AWS Amplify
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy async, Alembic
Auth	Amazon Cognito (ID token JWT). Dev auth only if <code>ENVIRONMENT=dev</code> + <code>DEV_AUTH_ENABLED=true</code>
RBAC	Cognito custom claims <code>custom:tenant_id</code> , <code>custom:role</code>
Database	PostgreSQL 16 local / Amazon RDS <code>ekba-dev-postgres</code> (<code>db.t4g.micro</code>)
Vector DB	Qdrant (local Docker or Qdrant Cloud). Collection <code>ekba_chunks</code>
Cache	Redis 7 sidecar (<code>redis:7-alpine</code>). No ElastiCache
Storage	Amazon S3 documents bucket (local MinIO via <code>S3_ENDPOINT_URL</code>)
Embeddings (ACTIVE)	Cohere <code>embed-multilingual-v3.0</code>

Embeddings (ALT)	Bedrock Titan <code>amazon.titan-embed-text-v2:0</code> (config switch)
LLM (ACTIVE)	Groq <code>openai/gpt-oss-20b</code>
LLM (ALT / fallback)	Bedrock Nova <code>us.amazon.nova-lite-v1:0</code> → fallback <code>us.amazon.nova-micro-v1:0</code>
Vision	Bedrock Nova Lite <code>describe_image</code>
STT (upload media)	Amazon Transcribe
STT (chat mic)	Browser Web Speech API (frontend)
TTS	NOT IMPLEMENTED
OCR service	NOT IMPLEMENTED — PDF uses pypdf text extract; images use vision
Agents	LangGraph
Deploy	Terraform, ECR, ECS Fargate, CodeDeploy blue-green, CloudFront, Amplify
CI/CD	<code>.github/workflows/ci.yml</code> , <code>deploy.yml</code> (OIDC) + <code>scripts/deploy.sh</code>
Observability	CloudWatch <code>/ekba/dev/service</code> ; LangSmith optional

3. Complete Architecture

```

User Browser
├─ Amplify (Next.js UI) — HTTPS
├─ CloudFront (API HTTPS edge)
│   └─ ALB ekba-dev-alb
│       ├── :80  production → TG ekba-dev-blue (or green after shift)
│       └── :8080 test      → TG ekba-dev-green (CodeDeploy health)
│           └─ ECS Fargate task ekba-dev (PUBLIC subnets,
assign_public_ip=true)
│               ├── api (FastAPI :8000)
│               ├── redis sidecar :6379
│               └─ qdrant sidecar (still started; UNUSED when
QDRANT_URL=Cloud)

FastAPI talks to:
Cognito (JWT JWKS) | RDS ekba-dev-postgres (private subnets)
S3 documents       | Secrets Manager ekba/dev/*
Qdrant Cloud       | Cohere Embed | Groq Chat
Bedrock (vision / optional providers) | Transcribe (media)
CloudWatch logs

CI/CD:
GitHub → Actions OIDC → ECR ekba-dev-backend → CodeDeploy ekba-dev / ekba-dev-dg

```

Network fact: There is **no NAT Gateway**. ECS tasks run in **public subnets** with public IPs. RDS runs in **private subnets**, reachable only from the tasks security group.

4. Request Flows

Auth principal: Bearer **Cognito ID token** → `verify_token` (`backend/app/core/auth.py`) → `RequestContext.tenant_id` / `role` from verified claims only (never from request body).

A) Login

1. Frontend → `POST /api/v1/auth/login`
2. Anonymous rate limit → Cognito `USER_PASSWORD_AUTH` (or local dev auth)
3. Returns ID token + refresh; stamps `users.last_login_at` ; audit `auth.login_succeeded`

B) Company creation

`POST /api/v1/platform/tenants` — requires `PlatformAdminUser` → row in `tenants` via control plane.

C) User invitation

- First company admin: `POST /api/v1/platform/tenants/{tenant_id}/admins`
- Inside company: `POST /api/v1/admin/users` (roles limited to `user` / `admin`)
- Writes Postgres user + Cognito invite (OTP password; app never stores passwords)

D–O) Other flows (summary)

Flow	Endpoint / entry	Key side effects
Upload	<code>POST /api/v1/documents</code>	S3 put + <code>documents</code> + <code>ingestion_jobs</code> + background ingest
Extract	<code>ingestion/extractors.py</code>	<code>pypdf</code> / <code>docx</code> / <code>csv</code> / <code>vision</code> / <code>Transcribe</code>
Chunk	<code>chunker.py</code>	512 tokens, overlap 64
Embed	<code>get_provider().embed</code>	Cohere (batches of 25 in ingestion)
Upsert	<code>vector.upsert_chunks</code>	Qdrant; <code>tenant_id</code> mandatory
RAG Q&A	<code>POST /api/v1/chat</code>	See §8
Feedback	<code>POST /api/v1/feedback</code>	<code>user_feedback</code>
Logout	<code>POST /api/v1/auth/logout</code>	Cognito global sign-out

5. Master Realistic Example

Company tenant + user "Rahul"

- POST /api/v1/auth/login → Cognito ID token (custom:tenant_id, custom:role)
- CloudFront → ALB :80 → ECS FastAPI
- POST /api/v1/documents (Employee Travel Policy.pdf)
- S3 key: {tenant_id}/{uuid}.pdf (AES256)
- Postgres documents + ingestion_jobs
- Background: pypdf extract → chunk 512/64
- Cohere embed-multilingual-v3.0 (1024-d)
- Qdrant Cloud collection ekba_chunks (tenant-scoped upsert)
- POST /api/v1/chat "What is the daily meal allowance?"
- injection scan → question embed → Redis semantic cache miss
- Qdrant search MUST filter tenant_id (+ exclude suspicious=true)
- score ≥ 0.35 → BM25+vector rerank → top_k=8
- SYSTEM_PROMPT + sources → Groq openai/gpt-oss-20b
- citation validation + output guardrail
- answer + citations → usage/messages → UI

6. Embedding Model

Item	Value
ACTIVE (AWS hybrid)	Cohere embed-multilingual-v3.0, dimension 1024
Code	CohereEmbeddingBackend in backend/app/services/ai/provider.py
Env	EMBED_PROVIDER, COHERE_API_KEY, COHERE_EMBED_MODEL, dim via BEDROCK_EMBEDDING_DIMENSION / embedding_dimension
Distance	COSINE in Qdrant
Query vs doc	Cohere input_type=search_query (1 text) else search_document
Ingestion batch	25 texts per embed call
Automatic fallback model	NOT IMPLEMENTED — failure raises UpstreamError
Config alternative	EMBED_PROVIDER=bedrock → Titan amazon.titan-embed-text-v2:0
Local stub	AI_PROVIDER=local hashed bag-of-words

7. LLM

Item	Value
ACTIVE	Groq openai/gpt-oss-20b

API	GROQ_API_BASE default <code>https://api.groq.com/openai/v1</code> + <code>/chat/completions</code>
Env	LLM_PROVIDER , LLM_MODEL , GROQ_API_KEY
RAG call	temperature 0.1 , max_tokens=max_output_tokens (1024)
Prompts	backend/app/services/rag/prompts.py
Bedrock optional	Primary <code>us.amazon.nova-lite-v1:0</code> ; fallback <code>us.amazon.nova-micro-v1:0</code> (Bedrock path only)
Groq automatic fallback	NOT IMPLEMENTED
Region (Bedrock)	<code>us-west-2</code>

8. RAG Pipeline

Exact order in `backend/app/services/rag/pipeline.py:answer_question` :

1. Authentication (route dependency)
2. Input validation (`validate_question`)
3. Prompt injection scan (`scan_input`) — may block
4. Embed question
5. Semantic cache (Redis, tenant-namespaced) — cache hit still runs output guardrail
6. Qdrant search — tenant filter built *inside* `vector.search` (cannot omit)
7. Relevance threshold: **0.35** (local stub uses **0.15**)
8. Over-fetch `top_k * 2` then BM25+vector rerank → `retrieval_top_k=8`
9. Context construction + system prompt
10. LLM chat
11. Citation validation (`[s1]` ... must match retrieved sources)
12. Output guardrail
13. Usage tracking (+ cache store if citations OK)

Reranking: local BM25 fusion (`rerank.py` , lexical weight 0.35) — not a hosted Cohere/Bedrock reranker.

Chunking: 512 / 64 tokens.

9. Qdrant

Setting	Value
---------	-------

Collection	ekba_chunks
Dimension	1024
Distance	COSINE
Point id	chunk_id (UUID)
Indexes	tenant_id, document_id, owner_id, department (KEYWORD); suspicious (BOOL)

```
{
  "id": "<chunk_uuid>",
  "vector": [ /* 1024 floats */ ],
  "payload": {
    "document_id": "...",
    "chunk_id": "...",
    "document_name": "Employee Travel Policy.pdf",
    "page_number": 3,
    "source_uri": "s3://ekba-dev-documents-.../tenant/uuid.pdf",
    "owner_id": "<cognito_sub>",
    "tenant_id": "<tenant_id>",
    "document_version": 1,
    "created_by": "...",
    "created_at": "ISO-8601",
    "text": "chunk text...",
    "modality": "text",
    "department": "hr",
    "section": null,
    "suspicious": false
  }
}
```

Code: backend/app/services/vector.py (ChunkPayload, ensure_collection, search, upsert_chunks, delete_document_chunks).

Manual inspect: Qdrant Cloud console → collection ekba_chunks → filter by tenant_id.

10. PostgreSQL / RDS

Instance: ekba-dev-postgres (private subnets). Models: backend/app/db/models.py.

Group	Tables
TENANT	tenants
AUTH/USER	users
DOCUMENT	documents
INGESTION	ingestion_jobs
CHAT	conversations, messages

FEEDBACK	<code>user_feedback</code>
USAGE	<code>request_usage</code>
AUDIT	<code>audit_events</code>
OTHER	<code>prompt_releases</code>

Inspect: AWS RDS console / Query Editor / `psql` using password from Secrets Manager — never print secret values.

11. S3

- Bucket: `ekba-dev-documents-<suffix>` (baseline Terraform)
- Key: `{tenant_id}/{uuid}.{ext}` via `files.build_storage_key`
- SSE: AES256
- Reads/deletes assert key prefix matches caller tenant
- Stores originals only — vectors in Qdrant; metadata in Postgres

Example key: `infinity-assurance/a1b2c3d4e5f6....pdf`

12. Cognito

- User pool name: `ekba-dev-users`
- App client name: `ekba-dev-web` (public SPA client)
- Custom attributes: `custom:tenant_id`, `custom:role`
- API uses **ID token** (carries custom claims)
- Verification: JWKS, issuer, audience, expiry
- `platform_admin` ↔ tenant id `platform` (enforced together)
- Passwords never stored/set by the application

13. Security

Layer	Protects	Where
Cognito JWT	Authentication	<code>core/auth.py</code>
RBAC deps	Authorization	<code>api/deps.py</code>
Tenant isolation	Cross-company data leak	JWT tenant → SQL / S3 prefix / Qdrant filter
Secrets Manager	API keys & DB password	ECS task secrets
VPC + SG	Network exposure	ALB <code>allowed_cidrs</code> ; DB only from tasks
CloudFront	HTTPS to clients	ALB remains HTTP

Injection / guardrails	Prompt abuse / unsafe output	security/injection.py , rag/guardrails.py
Rate limits	Abuse	Redis core/ratelimit.py
Upload safety	Malicious files	extension + magic bytes + size
OIDC deploy	No long-lived AWS keys in Actions	ekba-dev-github-deploy

14. ALB

Internet → CloudFront → ALB :80 → Target Group (blue/green) → ECS :8000 FastAPI
ALB :8080 (test) → green TG before traffic shift

- Names: ekba-dev-alb , ekba-dev-blue , ekba-dev-green
- Health check path: /api/v1/health
- If a task dies: target becomes unhealthy; ALB stops sending traffic; CodeDeploy can auto-rollback on 5xx alarm

15. Blue-Green Deployment

```
git push aws-deployment
→ GitHub Actions CI
→ Docker build → ECR :{git_sha}
→ New ECS task definition revision (image swap)
→ CodeDeploy app ekba-dev / group ekba-dev-dg
→ Green health-checked on :8080
→ Traffic shifts to green (CodeDeployDefault.ECSAllAtOnce)
→ Blue kept for rollback_window_minutes, then terminated
```

Infra / env / secret changes still go through ./scripts/deploy.sh (GHA does **not** apply Terraform for envs/dev).

scripts/rollback.sh : CodeDeploy traffic rollback is **NOT IMPLEMENTED** (placeholder TODO).

16. GitHub Actions

Workflow	Trigger	What it does
ci.yml	push main/develop; PRs; called by deploy	lint, pytest, frontend, terraform validate, docker build
deploy.yml	push aws-deployment (backend paths) or manual	OIDC → ECR → CodeDeploy → verify via API_URL

17. Terraform & Lifecycle Scripts

State	Lifecycle	Contents
envs/baseline	Protected	Cognito, ECR, S3 docs, Secrets containers, budget, OIDC role, audit log
envs/cost-guard	Protected	Kill-switch Lambda / budgets / SNS
envs/dev	Ephemeral	VPC, ALB, ECS, RDS, CodeDeploy
envs/frontend	Persistent	Amplify + CloudFront API

- `deploy.sh` — preflight, local tests, build/push, terraform apply **dev**, CodeDeploy, verify
- `destroy.sh` — snapshot RDS → destroy **only** dev state
- `deploy-frontend.sh` — Amplify + CloudFront + wire CORS/SG
- `verify.sh` — read-only health/readiness checks

18. Redis

- URL: `redis://localhost:6379/0` (sidecar; no persistence)
- Rate-limit keys: `ekba:rl:{bucket}:{tenant}:{user}`
- Semantic cache: `ekba:sc:idx:{tenant}` , `ekba:sc:e:{tenant}:{digest}`
- TTL: `semantic_cache_ttl_seconds=3600` ; similarity gate **0.95**
- Does **not** survive destroy / task replacement

19. CloudWatch

- Service logs: `/ekba/dev/service`
- Audit log group (baseline): `/ekba/dev/audit`
- Alarms: `ekba-dev-5xx` (CodeDeploy auto-rollback), `ekba-dev-latency`
- Billing alarm `ekba-dev-spend` : **NOT created in us-west-2** (us-east-1 only in Terraform)

20. Secrets Manager (names only)

Secret name	Injected as
<code>ekba/dev/backend/database-url</code>	RDS password → <code>DB_PASSWORD</code>
<code>ekba/dev/backend/qdrant-api-key</code>	<code>QDRANT_API_KEY</code>
<code>ekba/dev/backend/dev-auth-secret</code>	<code>DEV_AUTH_SECRET</code>
<code>ekba/dev/ai/langsmith-api-key</code>	<code>LANGSMITH_API_KEY</code>

ekba/dev/ai/cohere-api-key	COHERE_API_KEY
ekba/dev/ai/groq-api-key	GROQ_API_KEY

Secrets Manager → IAM execution role GetSecretValue → ECS task secrets → pydantic Settings → app

21. API Endpoint Map

Global prefix: /api/v1 (backend/app/main.py)

Method	Endpoint	Purpose	Auth
GET	/health	Liveness	No
GET	/health/ready	Postgres+Redis+Qdrant	No
GET	/me	Current principal	Yes
POST	/auth/login	Sign-in	No
POST	/auth/new-password	First login password	No
POST	/auth/refresh	Refresh session	No
POST	/auth/forgot-password	Reset start (always 202)	No
POST	/auth/confirm-password-reset	Reset confirm	No
POST	/auth/logout	Sign-out	Yes
POST	/documents	Upload	Yes
GET	/documents	List	Yes
GET	/documents/{id}	Get	Yes
GET	/documents/{id}/status	Ingestion status	Yes
GET	/documents/{id}/download	Download	Yes
DELETE	/documents/{id}	Delete	Yes
POST	/chat	RAG Q&A	Yes
POST	/feedback	Feedback	Yes
GET	/agents/workflows	List workflows	Yes
POST	/agents/run	Run agent	Yes
*	/admin/*	Company admin console	admin
*	/platform/*	Control plane	platform_admin

22. Data Lifecycle

```
USER → COGNITO JWT → FastAPI
  → Postgres (users, documents, jobs, messages, usage, audit)
  → S3 (original file bytes)
  → Extract / Chunk → Cohere Embed → Qdrant vectors
USER question → Embed → Qdrant(+tenant) → Groq → Citations → UI
  \ Redis (rate limit + semantic cache)
```

23. Destroy → Deploy Lifecycle

`./scripts/destroy.sh` (dev only): create manual RDS snapshot → terraform destroy **dev** state.

Survives: baseline (Cognito, ECR, S3 bucket, Secrets, budget), cost-guard, frontend (Amplify/CloudFront), state bucket, RDS snapshots, Qdrant Cloud data (external).

`./scripts/deploy.sh` : recreate ephemeral stack → restore newest RDS snapshot → ECS/CodeDeploy → verify.

`./scripts/deploy-frontend.sh` : Amplify + CloudFront wiring.

24. Interview Speeches

A) 30 seconds

"I built a multi-tenant enterprise RAG assistant: Cognito auth, FastAPI on ECS Fargate, documents in S3, vectors in Qdrant with mandatory tenant filters, embeddings via Cohere, answers via Groq, Postgres for metadata, Redis for cache and rate limits, and blue-green releases with CodeDeploy under a hard AWS cost ceiling."

B) 2 minutes

Add the onboarding hierarchy (platform → company admin → users), the ingestion pipeline (extract → chunk → embed → upsert), guardrails and citation validation, and the edge path CloudFront → ALB → ECS. Mention hybrid providers are config-driven so RAG code does not hardcode one LLM or embedding vendor.

C) 5 minutes

Walk login JWT claims → upload key design → chunk/embed/upsert → RAG stages including BM25 rerank → blue-green listeners :80/:8080 → destroy/snapshot lifecycle and what is protected.

D) 10 minutes

Deep-dive tenant isolation invariants, `RoutingProvider`, Secrets Manager injection, OIDC CI vs `deploy.sh` Terraform path, and honestly call out NOT IMPLEMENTED items (TTS, dedicated OCR, `rollback.sh` traffic shift).

25. Interview Q&A

Question	Strong answer
Why Qdrant?	Payload filters + COSINE + tenant index; filter is built inside <code>vector.search</code> so callers cannot omit it.
Why embeddings?	Semantic retrieval over policies; same model for index and query.
Why RAG?	Ground answers in company docs; citations validated against retrieved chunks.
Why Cohere?	Configured live embed path (<code>embed-multilingual-v3.0</code> , 1024-d); Titan remains a config alternative.
Why PostgreSQL?	Relational tenancy, jobs, audit, usage — durable across task replacement via RDS snapshots.
Why S3?	Durable originals with tenant-prefixed keys generated server-side.
Why Cognito?	Managed IdP; app never stores or sets passwords.
Why Redis?	Ephemeral rate limiting + semantic cache; cheap sidecar, no ElastiCache cost.
Why ALB / ECS / CodeDeploy?	Fargate + blue-green with a dedicated test listener before traffic shift.
Why CloudFront / Amplify?	HTTPS UI/API for users while ALB stays HTTP-only.
Tenant isolation?	JWT tenant only → SQL filters, S3 prefix checks, mandatory Qdrant tenant filter.
ECS crash?	Target health fails; traffic stops; new task; 5xx alarm can auto-rollback deploy.
Manual rollback script?	<code>rollback.sh</code> traffic shift is not implemented yet; CodeDeploy auto-rollback is.
Where are secrets?	AWS Secrets Manager under <code>ekba/dev/...</code> — never in git or frontend.
Cost control?	No NAT/EKS/ElastiCache; destroy after demo; gross spend guard near \$18.

26. Final Master Flow

RUNTIME

User → Amplify → CloudFront → ALB → ECS FastAPI
→ Cognito verify → Postgres / S3 / Qdrant / Redis
→ Cohere (embed) → Groq (chat) → Answer + citations

DEPLOY

GitHub → Actions (OIDC) → ECR → CodeDeploy blue/green → ALB traffic shift

27. File / Code References

Topic	Path
App entry / routes	<code>backend/app/main.py</code>
Config / providers	<code>backend/app/core/config.py</code>
JWT / RBAC	<code>backend/app/core/auth.py</code> , <code>api/deps.py</code>
Auth / platform / admin APIs	<code>api/v1/auth.py</code> , <code>platform.py</code> , <code>admin.py</code>
Documents / chat	<code>api/v1/documents.py</code> , <code>chat.py</code>
RAG	<code>services/rag/pipeline.py</code> , <code>guardrails.py</code> , <code>cache.py</code> , <code>rerank.py</code> , <code>prompts.py</code>
Ingestion	<code>services/ingestion/pipeline.py</code> , <code>extractors.py</code> , <code>chunker.py</code>
AI backends	<code>services/ai/provider.py</code> , <code>registry.py</code> , <code>transcribe.py</code>
Qdrant / S3	<code>services/vector.py</code> , <code>services/storage.py</code> , <code>security/files.py</code>
Models	<code>db/models.py</code>
AWS AI env	<code>infra/terraform/envs/dev/main.tf</code>
Blue-green	<code>infra/terraform/modules/service/main.tf</code>
Baseline secrets	<code>infra/terraform/envs/baseline/main.tf</code>
CI/CD	<code>.github/workflows/ci.yml</code> , <code>deploy.yml</code>
Lifecycle	<code>scripts/deploy.sh</code> , <code>destroy.sh</code> , <code>deploy-frontend.sh</code> , <code>rollback.sh</code>

28. NOT IMPLEMENTED (say this honestly in interviews)

- TTS (text-to-speech)
- Dedicated OCR service (Textract / Tesseract) — PDF is pypdf text extraction
- `scripts/rollback.sh` CodeDeploy traffic shift (placeholder)
- ElastiCache, NAT Gateway, EKS, EFS
- Automatic embedding-model failover (config switch only)
- Automatic Groq→Bedrock failover on quota errors (switch via `LLM_PROVIDER`)